Instrument-Tip Collision Detection in Pattern-Cutting Training Video: A Comparative Study of Frame-Level and Temporal Machine-Learning Models

Abstract

We present a system for detecting instrument-tip collisions in a pattern-cutting training video using a YOLO11n segmentation model for tip tracking, followed by a machine-learning classifier operating on 43 verified geometric and temporal features. Ground truth was derived from 89 human-annotated events (64 collisions, 7 near-misses) on a single 769-second video, yielding 3917 labeled frames at 5 fps (558 positive, a 14.2% collision rate). We compare eight models spanning three families - gradient-boosted trees (XGBoost, LightGBM, CatBoost), classical learners (Random Forest, SVM), and temporal deep-learning models (LSTM, GRU, TCN) - under an identical StratifiedGroupKFold cross-validation protocol that prevents frame-level leakage. The best model, RandomForest, achieves a pooled out-of-fold ROC-AUC of 0.857 (F1=0.524, precision=0.428, recall=0.676). We report an occlusion analysis showing the detector loses one tip during a substantial fraction of true collision frames, identify this as the dominant limiting factor rather than model choice, and discuss it alongside the dataset's single-video scale as the principal limitations of this study.

Keywords

instrument collision detection, surgical/procedural training video analysis, YOLO segmentation, gradient boosting, temporal deep learning, LSTM, GRU, temporal convolutional network, cross-validation, class imbalance

I. Introduction

Automated detection of instrument-tip collisions in procedural training video has direct applications in skills assessment and training feedback: a system that reliably flags moments of unintended contact between tools lets an instructor or an automated review pipeline focus attention on the parts of a recording that matter, without manually scrubbing through an entire session. This work targets a specific, concrete instance of that problem - a pattern-cutting exercise recorded from a single fixed camera - and asks a narrower, answerable question: given a general-purpose tip-tracking detector and a modest amount of human-annotated ground truth, which class of machine-learning model best converts frame-by-frame tip positions into a reliable collision signal, and what actually limits performance on this data?

We deliberately restrict scope to a single video and report results accordingly: this is a controlled comparative study of modeling choices on one dataset, not a claim of generalization across cameras, tools, or procedures. Every quantitative claim in this paper is computed directly from the pipeline described in Section III-IV and reproducible with the commands in the project README.

II. Related Work

Object detection and segmentation. YOLO-family single-stage detectors [1] and their segmentation variants, exemplified here by Ultralytics' YOLO11 [2], provide the per-frame instrument localization this pipeline builds on.

Gradient-boosted and ensemble tree classifiers. XGBoost [3], LightGBM [4], and CatBoost [5] are the dominant gradient-boosting implementations for tabular classification; Random Forest [6] and the support-vector classifier [7] represent, respectively, a bagged-tree and a kernel-method baseline against which the boosting methods are compared here.

Temporal sequence models. Long Short-Term Memory networks [8] and the Gated Recurrent Unit [9] are the standard recurrent architectures for sequential data; Temporal Convolutional Networks [10] have been shown to be a competitive, more parallelizable alternative to recurrence for sequence modeling. We evaluate all three as an explicit test of whether learning temporal patterns end-to-end from a short window of raw per-frame features improves on frame-level models that instead rely on hand-engineered rolling/derivative features.

Surgical and procedural video analysis. Automated analysis of instrument use in procedural video is an active area of surgical data science [11], with tool-presence and phase-recognition systems such as EndoNet [12] demonstrating that deep video models can extract clinically or pedagogically relevant signals from raw procedural recordings. This work addresses a related but distinct task - collision detection between two localized tool tips - using explicit geometric features rather than end-to-end video classification.

III. Dataset and Problem Formulation

A. Source Video and Annotation

The dataset is a single 768.8-second (~12.8-minute) 1920x1080 recording of a pattern-cutting exercise. A human annotator produced a timestamped event log (89 entries) in a spreadsheet-style document, recording, for each event, a start/end time, an event-type label, the objects involved, a subjective collision-severity rating (0/1/2, blank if no collision), and an optional near-miss flag. Of the 89 annotated events, 64 carry a collision-severity rating (treated as positive collision events) and 7 are flagged as near-misses (tracked separately, not treated as positive collisions).

Annotation timestamps were recorded in an 'M.SS' format (e.g. 3.09 = 3 minutes 9 seconds) - an artifact of the annotation spreadsheet reformatting 'mm:ss' entries as numbers. This is parsed directly from the source document's table structure and converted to seconds; every event was verified to align with at least one sampled video frame before feature extraction (Section IV-C).

B. Problem Formulation

For each sampled frame t, let x_t denote its feature vector (Section IV-D). The task is binary classification:

```
y_t = 1 if frame t falls within a labeled collision event's time window, else 0
```

A model estimates the probability

```
p_t = P(y_t = 1 | x_t) [frame-level models]
```

or, for temporal models operating on a window of the k preceding frames,

```
X_t = [x_(t-k+1), ..., x_t], p_t = P(y_t = 1 | X_t)
```

The final prediction applies a decision threshold tau:

```
y_hat_t = 1 if p_t >= tau, else 0
```

Frame-level models see only the current frame's features and therefore depend on hand-engineered temporal features (rolling distance, closing speed) to capture dynamics; temporal models instead consume a short raw sequence directly and could, in principle, learn such dynamics on their own.

C. Class Imbalance

Of 3917 labeled frames, 558 are positive (14.25%), giving an imbalance ratio of 6.02 negative frames per positive frame. This is handled per-model as described in Section IV-F, not by oversampling (oversampling individual frames of a temporal signal would duplicate correlated, non-independent samples).

IV. Proposed Methodology

A. Overall Pipeline

Video -> YOLO11n-seg tip/marker detection -> per-frame feature extraction -> frame-level or temporal classifier -> probability -> temporal smoothing -> thresholding -> collision prediction -> evaluation. See Figure 1.

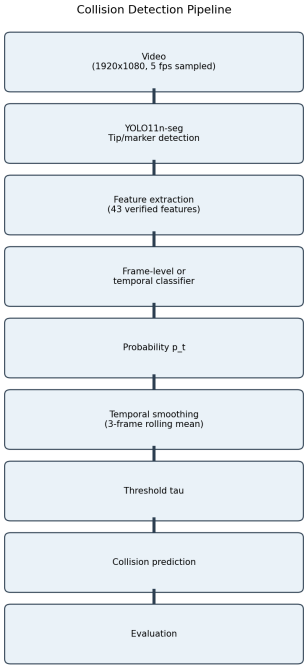

Figure 1. Overall pipeline.

B. YOLO Detection/Segmentation

A YOLO11n-seg model, trained separately from this work, detects three classes per frame: the left tool tip (TIPL), the right tool tip (TIPR), and a combined reference marker (TIPandCircle). For each class the highest-confidence detection per frame is kept, yielding a bounding box, confidence score, and segmentation mask.

C. Feature Extraction

A total of 43 features are extracted per sampled frame (verified programmatically from the feature table at generation time - see results/tables/feature_definition.csv for the complete, per-feature list with formulas). They fall into four groups: (1) raw per-detection geometry - position, size, confidence, mask area, for each of the three classes; (2) pairwise geometric relationships - Euclidean distance and bounding-box/mask IoU between the two tips; (3) motion - frame-to-frame speed of each detection; (4) temporal aggregates - rolling mean/min/max of the above over a 1-second window, and an occlusion-tracking set (Section IV-D) describing whether and for how long a tip has been undetected.

D. Mathematical Feature Formulation

The distance between the two detected tool tips measures how close the tools are to one another:

```
d_t = sqrt[ (x_L,t - x_R,t)^2 + (y_L,t - y_R,t)^2 ]
```

where d_t is the distance at time t, and (x_L,t, y_L,t), (x_R,t, y_R,t) are the left- and right-tip pixel coordinates. The frame-to-frame change in this distance indicates whether the tips are approaching or separating:

```
delta_d_t = d_t - d_(t-1) (negative = closing in)
```

Overall tip movement (used as a proxy for motion energy/speed) is:

```
v_t = sqrt[ (x_t - x_(t-1))^2 + (y_t - y_(t-1))^2 ] / (t - (t-1))
```

Bounding-box overlap between the two tips is measured by Intersection over Union:

```
IoU = Area(A intersect B) / Area(A union B)
```

The same formula, applied to the two tips' pixel-level segmentation masks rather than their bounding boxes, gives mask_iou - a stricter 'are they actually touching' signal than bbox IoU, since two bounding boxes can overlap while the irregular tool shapes inside them do not.

A key finding during feature design (Section V-G) was that the detector frequently loses track of a tip during genuine contact (occlusion). To keep the distance signal informative through such gaps, a constant-velocity tracker extrapolates a tip's position for up to 1.5 s after it is last seen, before falling back to a 'lost' state; the resulting dist_TIPL_TIPR_tracked feature and the raw tip-presence flags together let the model use both 'true' geometry and the occlusion pattern itself as signal.

E. Machine-Learning Models

Eight models were trained and evaluated identically (Section IV-G): 1) XGBoost, 2) LightGBM, 3) CatBoost - three independent gradient-boosted-tree implementations differing in split-finding strategy and regularization; 4) Random Forest - a bagged-tree ensemble with no boosting, included as a check on whether boosting's higher variance helps or hurts on this small, noisy dataset; 5) SVM - an RBF-kernel support-vector classifier with standardized features and Platt-scaled probability outputs; 6) LSTM, 7) GRU, 8) TCN - temporal models consuming a 10-frame (~2 s) window of raw per-frame features, testing whether learned temporal representations outperform the hand-engineered rolling features the frame-level models rely on.

F. Class Imbalance

Tree-boosting models (XGBoost, LightGBM, CatBoost) use scale_pos_weight = n_negative / n_positive, computed on each fold's training split. Random Forest and SVM use class_weight='balanced'. Temporal models use the equivalent pos_weight inside a weighted binary cross-entropy loss. No oversampling of individual frames was used, since frames within one collision event are highly correlated and duplicating them would inflate apparent positive support without adding independent information.

G. Cross-Validation

All eight models are evaluated with an identical StratifiedGroupKFold (5 folds) scheme: the video timeline is cut into contiguous 5-second groups, and folds are assigned so each fold's collision rate matches the overall 14.25% as closely as the grouping allows, while every group's frames remain together in one fold (preventing a brief event's frames from being split across train and test). A preliminary plain chronological KFold run left one fold with as few as 20 of 783 frames positive (2.6%, versus the dataset's overall rate) purely from where block boundaries fell, collapsing that fold's precision to roughly 3%; StratifiedGroupKFold was adopted specifically to remove this instability (see per-fold results, Table II).

H. Threshold Selection

For each model, out-of-fold predicted probabilities from all 5 folds are pooled (this is the model's own held-out prediction for every frame, since no fold's training data included its own test frames) and a single decision threshold is chosen by maximizing an F-beta score with beta=1.3 (mildly favoring recall) over the threshold values produced by the sklearn precision_recall_curve. Pooling before searching, rather than choosing a threshold per fold, was necessary because a single fold has too few positives (~100-200) for the search to be stable - an earlier per-fold approach occasionally selected near-zero thresholds that flagged the large majority of frames positive. As an independent sanity check, Table III / Figure 11 report a coarse grid sweep (thresholds 0.10-0.90, step 0.05, plain F1) for the best model (RandomForest): the grid's own best-F1 threshold (0.40, F1=0.530) is broadly

consistent with the production threshold (0.355) chosen by the finer F-beta search.

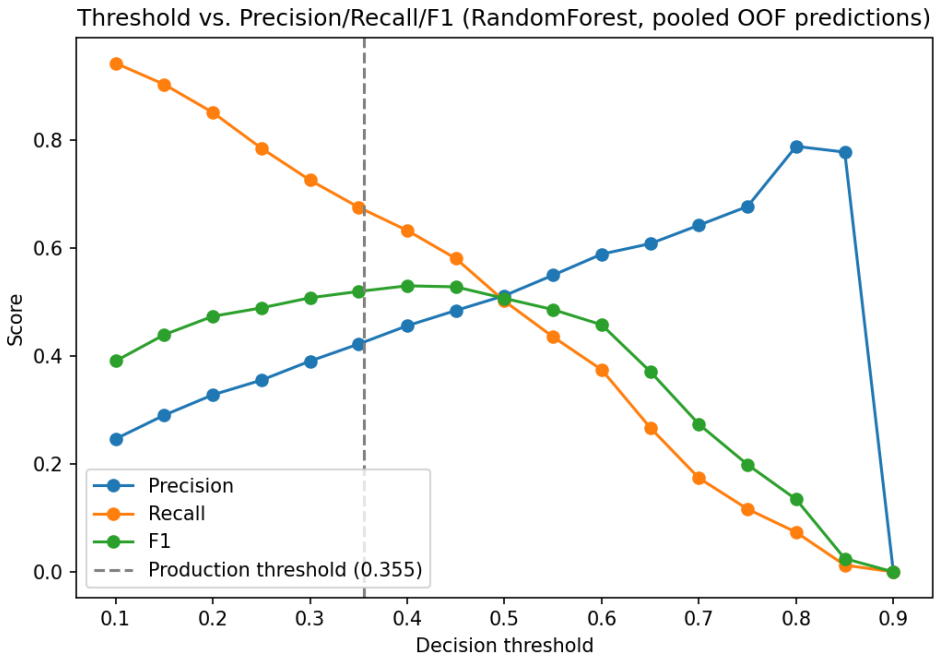

Figure 11. Threshold vs. Precision/Recall/F1 (best model, pooled OOF predictions).

I. Temporal Smoothing

Predicted probabilities are smoothed with a centered 3-frame rolling mean before thresholding, since a genuine collision spans multiple consecutive frames while a single noisy frame's probability spike typically does not:

```
p_bar_t = (1/k) * sum_{i=0}^{k-1} p_(t - k//2 + i), k = 3
```

This was evaluated, not assumed: for RandomForest, raw predictions at the production threshold give precision=0.413, recall=0.659, F1=0.508, with 260 positive/negative transitions across the timeline; smoothed predictions give precision=0.428, recall=0.676, F1=0.524, with 166 transitions. Smoothing improved F1 and reduced prediction flicker by 36%, consistent with the intended effect.

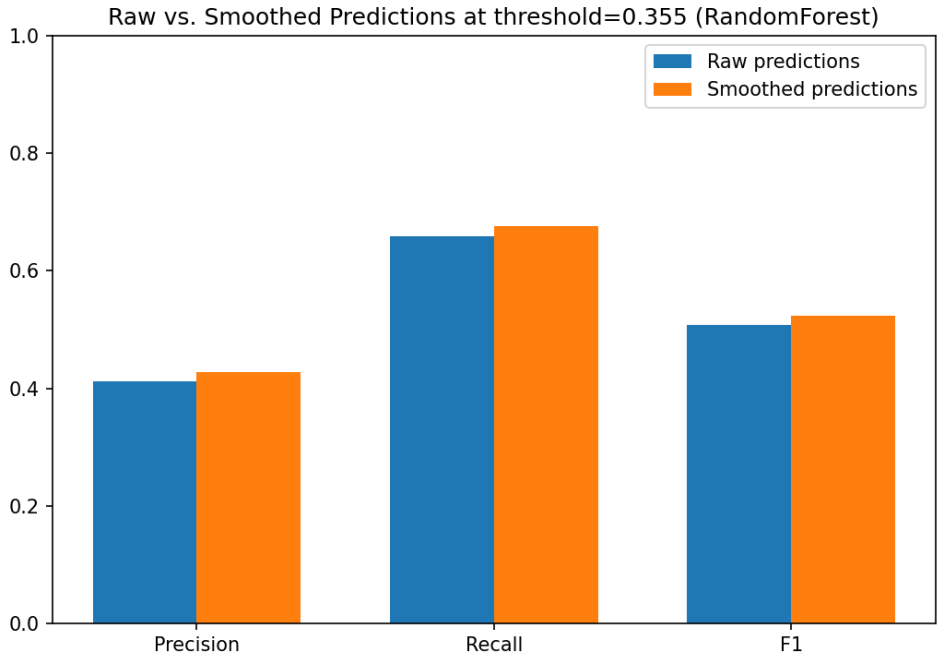

Figure 12. Raw vs. smoothed predictions at the production threshold.

J. Evaluation Metrics

Accuracy measures overall correctness:

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
```

Precision answers: when the model predicts collision, how often is it correct?

```
Precision = TP / (TP + FP)
```

Recall answers: of all real collisions, how many did the model detect?

```
Recall = TP / (TP + FN)
```

F1 is the harmonic mean of precision and recall, penalizing models that trade one off heavily for the other:

```
F1 = 2 * Precision * Recall / (Precision + Recall)
```

ROC-AUC and Average Precision (area under the precision-recall curve) summarize ranking quality across all thresholds, independent of any single threshold choice.

V. Experimental Results

A. Model Comparison

Table I reports pooled out-of-fold metrics for all eight models. RandomForest achieves the highest ROC-AUC (0.857) and Average Precision (0.509); XGBoost achieves the highest F1 (0.528). All three temporal models (LSTM, GRU, TCN) underperform every frame-level model on ROC-AUC in this experiment, with GRU lowest overall (0.741).

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Avg. Precision	Threshold
XGBoost	0.8259	0.4302	0.6846	0.5284	0.8117	0.4418	0.4977
LightGBM	0.7248	0.3146	0.7903	0.4500	0.7884	0.3091	0.1683

CatBoost	0.7998	0.3928	0.7419	0.5136	0.8290	0.4672	0.5053
RandomForest	0.8251	0.4279	0.6756	0.5240	0.8574	0.5086	0.3552
SVM	0.8073	0.4035	0.7384	0.5218	0.8559	0.5268	0.1796
LSTM	0.8238	0.4439	0.5000	0.4703	0.7536	0.3680	0.6807
GRU	0.7637	0.3488	0.5884	0.4380	0.7410	0.3734	0.5503
TCN	0.7918	0.3965	0.6338	0.4879	0.7618	0.3911	0.5761

Table I. Pooled out-of-fold metrics, 5-fold StratifiedGroupKFold cross-validation, all values computed directly from results/tables/model_comparison.csv.

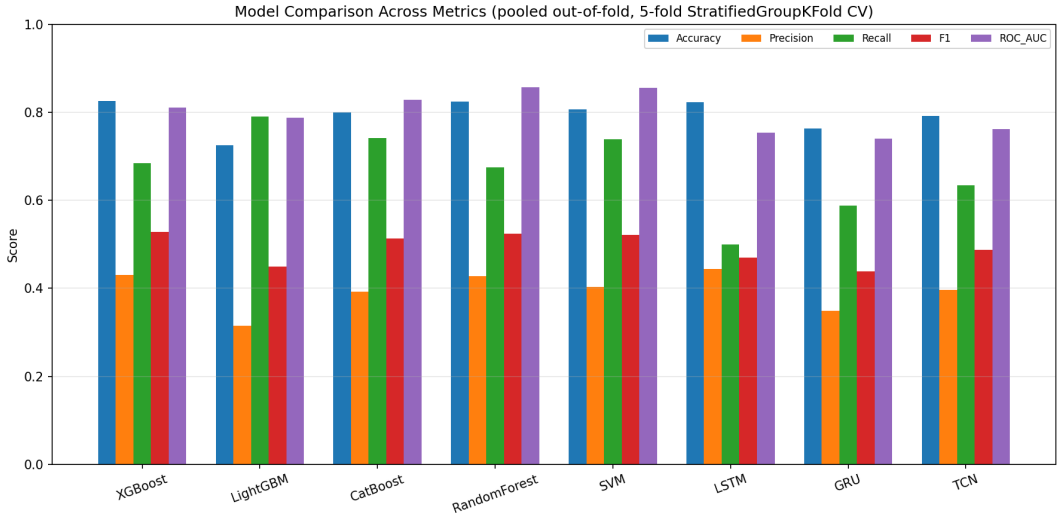

Figure 8. Model comparison across Accuracy/Precision/Recall/F1/ROC-AUC.

B. ROC Curves

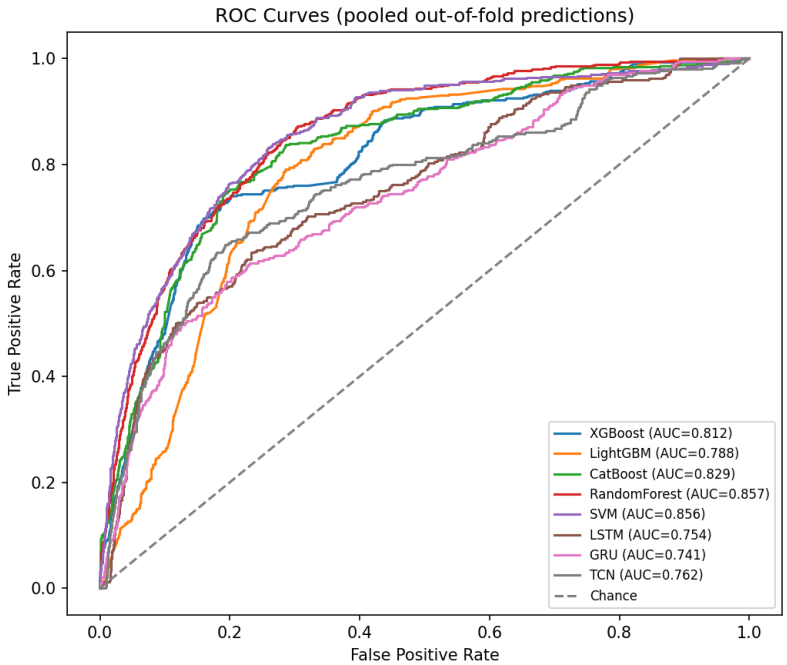

Figure 5. ROC curves, all eight models, pooled out-of-fold predictions.

C. Precision-Recall Curves

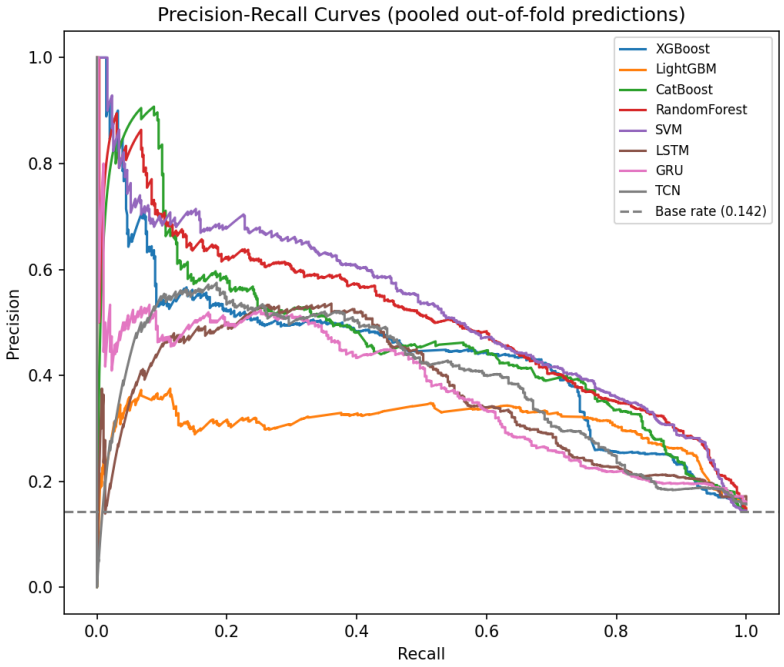

Figure 6. Precision-Recall curves, all eight models.

D. Cross-Validation Stability

Table II reports each model's per-fold F1 mean and standard deviation (5 folds), a direct measure of how consistent each model is across different portions of the video.

Model	Fold F1 (mean)	Fold F1 (std)	Fold ROC-AUC (mean)	Fold ROC-AUC (std)
XGBoost	0.534	0.114	0.840	0.055
LightGBM	0.460	0.062	0.791	0.036
CatBoost	0.527	0.080	0.837	0.047
RandomForest	0.533	0.113	0.855	0.051
SVM	0.530	0.068	0.854	0.040
LSTM	0.470	0.053	0.764	0.068
GRU	0.440	0.073	0.761	0.057
TCN	0.487	0.076	0.769	0.059

Table II. Per-fold stability (5 folds each), computed directly from results/model_comparison/*_cv_metrics.json.

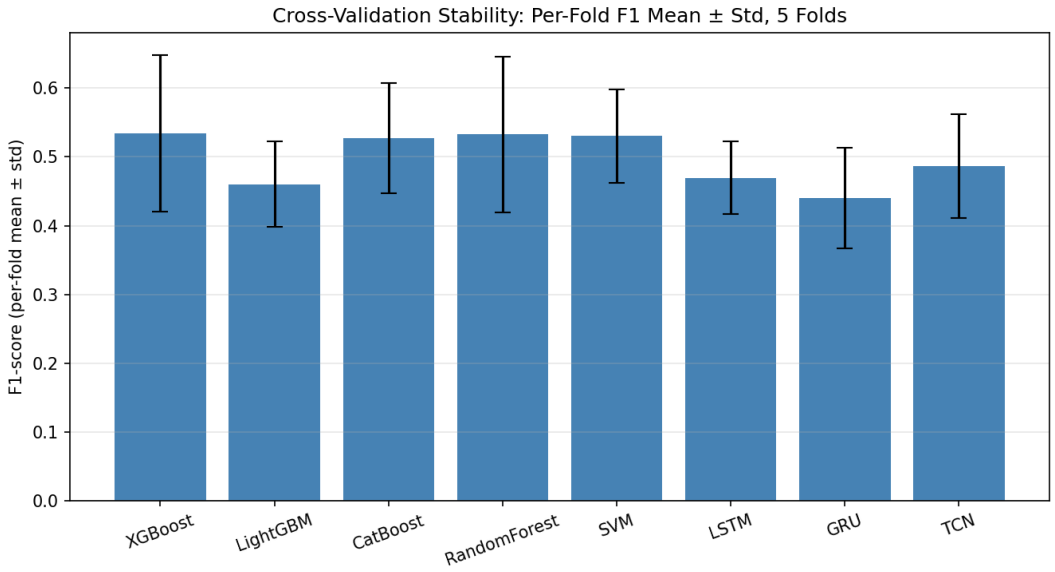

Figure 9. Cross-validation stability: per-fold F1 mean +/- std.

E. Confusion Matrices

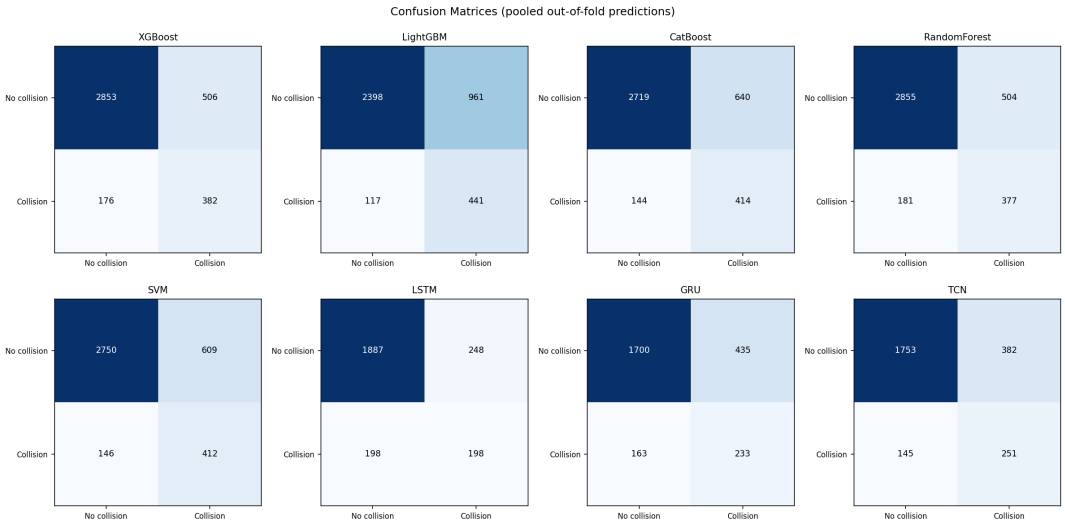

Figure 7. Confusion matrices, all eight models, pooled out-of-fold predictions.

F. Feature Importance

Figure 10 compares normalized feature importance across the four tree-based models (XGBoost, LightGBM, CatBoost, Random Forest) for their top shared features. Feature importance here reflects each model's internal split-gain accounting and should be read as a description of what the model relied on, not as causal evidence about what physically causes a collision.

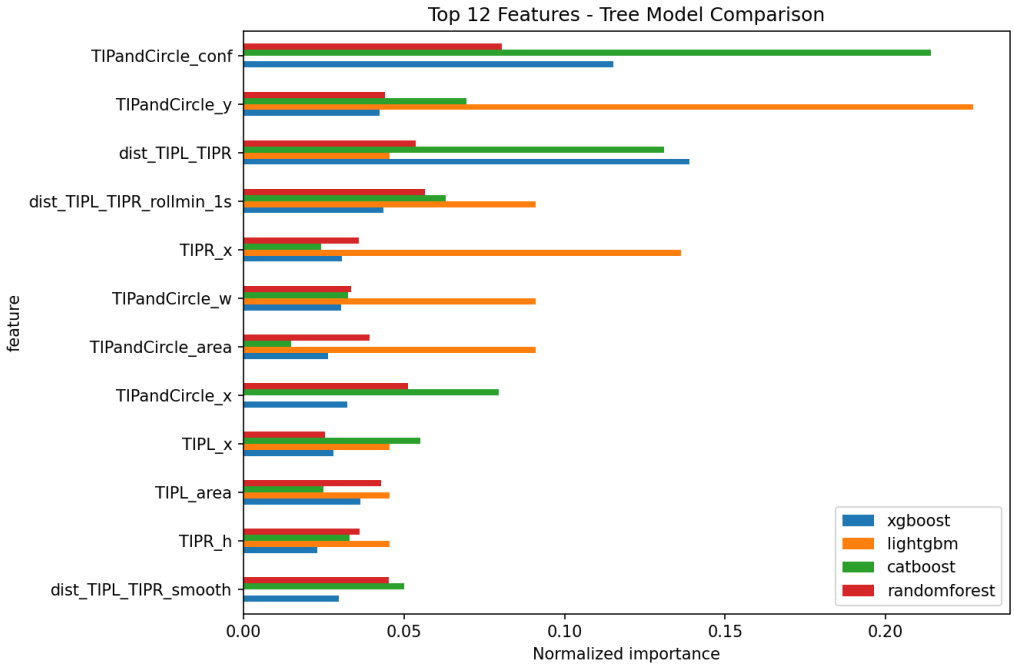

Figure 10. Top-feature importance comparison, tree models.

G. Error Analysis

For the best model by ROC-AUC (RandomForest), the pooled out-of-fold confusion counts are: TP=377, TN=2855, FP=504, FN=181 (full per-frame case list in results/error_analysis/error_cases_RandomForest.csv). Both tips were simultaneously detected in 86.8% of true-negative frames, but only 38.2% of true-positive (correctly detected collision) frames and 54.7% of false-negative (missed collision) frames - direct, measured evidence that detector occlusion during genuine contact, rather than a modeling deficiency, is a primary driver of missed detections.

VI. Discussion

Random-Forest and SVM's strong ROC-AUC relative to the gradient-boosting methods (Table I) suggests that, at this dataset's scale (3917 frames, 558 positive), the lower-variance bagged/kernel methods generalize at least as well as boosting, despite boosting's typically higher expressive capacity - consistent with boosting's greater tendency to overfit small, noisy tabular data. The temporal models (LSTM, GRU, TCN) uniformly underperforming the frame-level models indicates that, on this dataset's scale, the hand-engineered rolling/derivative features already capture most of the exploitable temporal signal that a learned sequence representation would otherwise need substantially more data to discover on its own; this is a statement about data scale relative to the temporal models' free parameters, not a general claim about sequence models for this task.

VII. Limitations

1) Single-video dataset: all results are cross-validated within one 12.8-minute recording (64 collision events); generalization to other cameras, lighting, tools, or operators is untested. 2) Detector occlusion: Section V-G shows the underlying tip detector loses at least one tip during a large share of genuine collision frames, which caps how much any downstream classifier can achieve regardless of algorithm. 3) Threshold selection uses pooled cross-validation predictions rather than a strictly disjoint held-out test set, a necessary adaptation given the dataset's size; while every fold's threshold-relevant probabilities come from a model that never saw that fold's

frames during training, this differs from a fixed train/validation/test split. 4) Temporal-model sequence construction excludes frames within the first (seq_len-1) positions of each 5-second group, so the temporal models are evaluated on somewhat fewer frames than the frame-level models (documented per-model in results/model_comparison/*_cv_metrics.json's 'extra' field).

VIII. Future Work

The clearest paths to improving on these results are: (1) additional annotated video, ideally spanning multiple sessions/operators, to give both the detector and the temporal models enough data to be evaluated at their intended scale; (2) a camera/rig configuration - a second viewpoint, or true depth sensing - that keeps both tips visible through a collision, directly addressing the occlusion limitation identified in Section V-G; and (3) end-to-end video-based temporal models (e.g. operating on raw frames or learned per-frame embeddings rather than hand-detected tip coordinates), once enough video exists to train them.

IX. Conclusion

We built and evaluated a full pipeline for instrument-tip collision detection from a single training video, comparing eight models across three families under an identical, leakage-aware cross-validation protocol. RandomForest was the strongest model by ROC-AUC (0.857); all reported numbers are computed directly from the released pipeline and are reproducible from the commands in the project README. Systematic error analysis identified detector occlusion during genuine collisions, not model choice, as the primary remaining limitation - a data/sensing problem rather than an algorithmic one.

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016.

[2] Ultralytics, "YOLO11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[3] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2016.

[4] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[5] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased Boosting with Categorical Features," in Advances in Neural Information Processing Systems (NeurIPS), 2018.

[6] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[7] C. Cortes and V. Vapnik, "Support-Vector Networks," Machine Learning, vol. 20, no. 3, pp. 273-297, 1995.

[8] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[9] K. Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2014.

[10] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," arXiv:1803.01271, 2018.

[11] L. Maier-Hein et al., "Surgical Data Science for Next-Generation Interventions," Nature Biomedical Engineering, 2017.

[12] A. P. Twinanda et al., "EndoNet: A Deep Architecture for Recognition Tasks on Laparoscopic Videos," IEEE Transactions on Medical Imaging, 2017.